ProjectMatch – Implementation Guide

A phase-by-phase plan (~10–12 weeks for a team of 2–4). Each phase ends with something demoable.

Phase	Weeks	Deliverable
0. Setup	0.5	Repo, tooling, CI, deployments skeleton
1. Auth & Profiles	1.5	Register/login, profile CRUD
2. Skills & Interests	1	Taxonomy, skill picker, availability
3. Projects & Teams	2	Listings, requests, team formation
4. AI Matching	2	Recommendations with explanations
5. Chat & Notifications	1.5	Real-time chat, in-app/email notifications
6. Task Management	1	Kanban board
7. Ratings & Admin	1.5	Peer reviews, admin dashboard
8. Hardening & Deploy	1	Tests, security, production deploy, demo

---

Phase 0 – Setup

```
mkdir projectmatch && cd projectmatch
npm create vite@latest client -- --template react
mkdir server && cd server && npm init -y
npm i express mongoose cors helmet dotenv bcryptjs jsonwebtoken cookie-
parser \
      zod express-rate-limit express-mongo-sanitize socket.io pino
morgan
npm i -D nodemon jest supertest mongodb-memory-server eslint prettier
```

`docker-compose.yml` (local services):

```
services:
  mongo:
    image: mongo:7
    ports: ["27017:27017"]
    volumes: [mongo_data:/data/db]
  redis:
    image: redis:7
    ports: ["6379:6379"]
volumes:
  mongo_data:
```

`server/src/app.js`

```
import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import cookieParser from 'cookie-parser';
import mongoSanitize from 'express-mongo-sanitize';
import { env } from './config/env.js';
import routes from './modules/index.js';
import { errorHandler } from './middleware/errorHandler.js';
import { globalLimiter } from './middleware/rateLimiter.js';

const app = express();
app.use(helmet());
app.use(cors({ origin: env.CLIENT_URL, credentials: true }));
app.use(express.json({ limit: '1mb' }));
app.use(cookieParser());
app.use(mongoSanitize());
app.use(globalLimiter);
app.get('/health', (_, res) => res.json({ ok: true }));
app.use('/api/v1', routes);
app.use(errorHandler);
export default app;
```

Phase 1 – Authentication & Profiles

Endpoints

Method	Route	Description
POST	`/auth/register`	Create account, send verification email
POST	`/auth/verify-email`	Verify token
POST	`/auth/login`	Returns access token + sets refresh cookie
POST	`/auth/refresh`	Rotate refresh token
POST	`/auth/logout`	Clear cookie, revoke token
POST	`/auth/forgot-password`, `/auth/reset-password`	Reset flow
GET	`/users/me`	Current profile
PATCH	`/users/me`	Update profile
GET	`/users/:id`	Public profile
GET	`/users`	Search/filter students

Token utilities

```
// utils/token.js
import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';

export const signAccess  = (user) =>
  jwt.sign({ sub: user._id, role: user.role }, env.JWT_ACCESS_SECRET, {
expiresIn: env.JWT_ACCESS_EXPIRES });
export const signRefresh = (user) =>
  jwt.sign({ sub: user._id }, env.JWT_REFRESH_SECRET, { expiresIn:
env.JWT_REFRESH_EXPIRES });
```

Auth middleware

```
// middleware/auth.js
import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';
import { ApiError } from '../utils/ApiError.js';
import User from '../modules/users/user.model.js';

export const requireAuth = async (req, _res, next) => {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return next(new ApiError(401, 'NO_TOKEN', 'Authentication
required'));
  try {
    const payload = jwt.verify(token, env.JWT_ACCESS_SECRET);
    const user = await User.findById(payload.sub).select('-
passwordHash');
    if (!user || !user.isActive) throw new Error();
    req.user = user;
    next();
  } catch {
    next(new ApiError(401, 'INVALID_TOKEN', 'Invalid or expired
token'));
  }
};

export const requireRole = (...roles) => (req, _res, next) =>
  roles.includes(req.user.role) ? next() : next(new ApiError(403,
'FORBIDDEN', 'Not allowed'));
```

Login service (core logic)

```
export async function login({ email, password }) {
  const user = await User.findOne({ email }).select('+passwordHash');
  if (!user || !(await bcrypt.compare(password, user.passwordHash)))
    throw new ApiError(401, 'BAD_CREDENTIALS', 'Invalid email or
password');
  if (!user.isVerified) throw new ApiError(403, 'UNVERIFIED', 'Verify
your email first');
  return { user, accessToken: signAccess(user), refreshToken:
signRefresh(user) };
}
```

Frontend

- Axios instance with `withCredentials: true`; interceptor retries once after `POST /auth/refresh` on 401.
- `ProtectedRoute` checks Redux `auth.user`; on app load call `/users/me` to hydrate.
- Registration form with validation (react-hook-form + zod).

Phase 2 – Skills & Interests

- `Skill` collection seeded from `seed/skills.json` (React, Node, Python, ML, Figma, DevOps, ...) with categories.
- `GET /skills?q=` (autocomplete), admin CRUD later.
- Profile editor: **SkillPicker** (search → select → level slider 1–5), interests as tags, preferred tech, availability (hours/week + time slots), `lookingFor` toggles.
- On profile save, enqueue `embedding.job` (see Phase 4) and compute `profileCompleteness` to nudge users.

Phase 3 – Projects, Requests & Teams

Endpoints

Method	Route	Description

POST/GET	`/projects`	Create / list with filters (type, skills, status, q, deadline)
GET/PATCH/DELETE	`/projects/:id`	Detail / update / delete (owner)
POST	`/projects/:id/apply`	Apply for a role
POST	`/requests`	Send invite
GET	`/requests?box=incoming	outgoing`
PATCH	`/requests/:id`	accept / decline / cancel
GET	`/teams/my`	My teams
GET	`/teams/:id`	Team detail (members only)
DELETE	`/teams/:id/members/:userId`	Remove / leave
PATCH	`/projects/:id/status`	Mark completed, etc.

Accept request (race-safe)

```
export async function acceptRequest(requestId, userId) {
  const session = await mongoose.startSession();
  try {
    await session.withTransaction(async () => {
      const reqDoc = await Request.findById(requestId).session(session);
      if (!reqDoc || reqDoc.status !== 'pending') throw new
ApiError(409, 'STALE', 'Request not pending');
      const acceptor = reqDoc.type === 'invite' ? reqDoc.to :
reqDoc.project.owner;
      if (String(acceptor) !== String(userId)) throw new ApiError(403,
'FORBIDDEN', 'Not allowed');

      // atomically claim a slot
      const updated = await Project.findOneAndUpdate(
        { _id: reqDoc.project, 'requiredRoles.role': reqDoc.role,
          $expr: { $lt: [{ $arrayElemAt: ['$requiredRoles.filled', 0] },
{ $arrayElemAt: ['$requiredRoles.slots', 0] }] } },
        { $inc: { 'requiredRoles.$.filled': 1 } },
        { session, new: true }
      );
      if (!updated) throw new ApiError(409, 'FULL', 'No slots left for
this role');

      await Team.updateOne({ _id: reqDoc.team },
        { $push: { members: { user: reqDoc.type === 'invite' ? reqDoc.to
: reqDoc.from, role: reqDoc.role } } },
        { session });
      reqDoc.status = 'accepted';
      await reqDoc.save({ session });
    });
  } finally { session.endSession(); }
  // after commit: notify, add to team conversation
}
```

> Transactions need a replica set (Atlas provides this; for local dev use a single-node replica set).
> Simplification: the `$expr` above is illustrative; in practice use `arrayFilters` or keep slots in a separate `RoleSlot` collection to make the atomic update simpler.

Frontend

- ProjectList with FilterBar, infinite scroll, skill chips
- ProjectForm: dynamic role rows (role name, skills, slots)
- Requests page with Accept/Decline buttons and optimistic UI

Phase 4 – AI Team Matching

4.1 Embeddings service

```
// matching/embedding.service.js
import { llm } from '../../services/llm.client.js';

export function profileToText(u) {
  const skills = u.skills.map(s => `${s.skill.name}(level
${s.level})`).join(', ');
  return `Skills: ${skills}. Interests: ${u.interests.join(', ')}. ` +
         `Preferred tech: ${u.preferredTech.map(t => t.name).join(',
')}. ` +
         `Experience: ${u.experience.map(e => e.title).join('; ')}.`;
}

export async function embedUser(userId) {
  const user = await User.findById(userId).populate('skills.skill
preferredTech');
  const vector = await llm.embed(profileToText(user));
  await User.updateOne({ _id: userId }, { embedding: vector,
embeddingUpdatedAt: new Date() });
}
```

Run `embedUser` through the BullMQ job after profile updates (debounced).

4.2 Pure scoring functions

```
// matching/scoring.js
import { W } from '../../config/matchWeights.js';

export const cosine = (a, b) => {
  let dot = 0, na = 0, nb = 0;
  for (let i = 0; i < a.length; i++) { dot += a[i]*b[i]; na += a[i]**2;
nb += b[i]**2; }
  return dot / (Math.sqrt(na) * Math.sqrt(nb) || 1);
};

export const skillCoverage = (requiredIds, cand) => {
  if (!requiredIds.length) return 0;
  const lvl = new Map(cand.skills.map(s => [String(s.skill), s.level /
5]));
  return requiredIds.reduce((sum, id) => sum + (lvl.get(String(id)) ||
0), 0) / requiredIds.length;
};

// share of currently-uncovered skills this candidate fills
export const complementarity = (gapIds, cand) => {
  if (!gapIds.length) return 0;
  const has = new Set(cand.skills.map(s => String(s.skill)));
  return gapIds.filter(id => has.has(String(id))).length /
gapIds.length;
};

export const availabilityOverlap = (a, b) => {
  const hours = Math.min(a.hoursPerWeek, b.hoursPerWeek) /
Math.max(a.hoursPerWeek, b.hoursPerWeek || 1);
  const slotsA = new Set(a.slots), shared = b.slots.filter(s =>
slotsA.has(s)).length;
  const slotScore = shared / Math.max(a.slots.length, 1);
  return 0.5 * hours + 0.5 * slotScore;
};

export const finalScore = (f) =>
  W.semantic * f.semantic + W.coverage * f.coverage + W.complement *
```

```
f.complement +
  W.availability * f.availability + W.reputation * f.reputation +
W.interest * f.interest;
```

`config/matchWeights.js`

```
export const W = { semantic: .35, coverage: .30, complement: .15,
availability: .10, reputation: .05, interest: .05 };
```

4.3 Orchestrator

```
// matching/matching.service.js
export async function recommendForProject(projectId, requester, { limit
= 10 } = {}) {
  const project = await
Project.findById(projectId).populate('requiredSkills');
  const team = await Team.findOne({ project: projectId });
  const memberIds = team.members.map(m => m.user);

  const covered = new Set((await User.find({ _id: { $in: memberIds }
})).flatMap(u => u.skills.map(s => String(s.skill))));
  const gap = project.requiredSkills.map(s => s._id).filter(id =>
!covered.has(String(id)));

  // Stage 1: retrieve (Atlas Vector Search; fallback = in-memory cosine
for small pools)
  const candidates = await User.aggregate([
    { $vectorSearch: { index: 'user_embedding', path: 'embedding',
queryVector: project.embedding,
                       numCandidates: 200, limit: 100,
                       filter: { isActive: true, _id: { $nin: memberIds
} } } },
  ]);

  // Stage 2: re-rank
  const ranked = candidates.map(c => {
    const features = {
      semantic: cosine(project.embedding, c.embedding),
      coverage: skillCoverage(project.requiredSkills.map(s => s._id),
c),
      complement: complementarity(gap, c),
      availability: availabilityOverlap(project.availability || {
hoursPerWeek: 10, slots: [] }, c.availability),
      reputation: (c.reputation?.avg || 3) / 5,
      interest: 0,
    };
    return { user: c, features, score: finalScore(features) };
  }).sort((a, b) => b.score - a.score).slice(0, limit);
```

```
  // Stage 3: explanations (cached, non-blocking failure)
  return Promise.all(ranked.map(async r => ({
    ...r, explanation: await explainMatch(project, r).catch(() => null),
  })));
}
```

4.4 LLM explanation (explanations only, ranking stays deterministic)

```
// matching/explain.service.js
export async function explainMatch(project, r) {
  const key =
`explain:${project._id}:${r.user._id}:${r.user.embeddingUpdatedAt?.getTi
me()}`;
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);

  const prompt = `You help students form teams. In 2 sentences, explain
why this candidate fits the project, ` +
    `and suggest one role. Return JSON {"reason": string,
"suggestedRole": string}.\n` +
    JSON.stringify({
      project: { title: project.title, requiredSkills:
project.requiredSkills.map(s => s.name) },
      candidate: { skills: r.user.skillNames, interests:
r.user.interests },
      scores: r.features,
    });

  const out = await llm.json(prompt);
  await redis.set(key, JSON.stringify(out), 'EX', 60 * 60 * 24);
  return out;
}
```

Never send names, emails or contact details to the LLM.

4.5 Endpoints

Method	Route	Description
GET	`/matching/teammates`	Recommendations for the current user (peer matching)
GET	`/matching/projects/:id/candidates`	Candidates for a project (owner only)
GET	`/matching/projects`	Projects recommended to the current user
POST	`/matching/feedback`	Log `viewed / invited / dismissed` for tuning

4.6 Frontend

`MatchCard` shows avatar, score bar, matched/missing skill chips, "Why this match" text and an **Invite** button. Add skeleton loading and a "refresh" button (rate-limited).

4.7 Evaluation

Create ~50 synthetic profiles with known "good" pairings; script computes precision@5 and compares weight sets. Track the invite acceptance rate in production.

Phase 5 – Real-Time Chat & Notifications

Socket server

```
// sockets/index.js
import { Server } from 'socket.io';
import jwt from 'jsonwebtoken';

export function initSockets(httpServer) {
  const io = new Server(httpServer, { cors: { origin: env.CLIENT_URL,
credentials: true } });

  io.use((socket, next) => {
    try {
      const payload = jwt.verify(socket.handshake.auth.token,
env.JWT_ACCESS_SECRET);
      socket.userId = payload.sub;
      next();
    } catch { next(new Error('unauthorized')); }
  });

  io.on('connection', (socket) => {
    socket.join(`user:${socket.userId}`);
    registerChatHandlers(io, socket);
    registerTaskHandlers(io, socket);
  });
  return io;
}
```

Chat handlers

```
socket.on('chat:join', async (conversationId, ack) => {
  const ok = await chatService.isParticipant(conversationId,
socket.userId);
  if (!ok) return ack?.({ error: 'FORBIDDEN' });
  socket.join(`conversation:${conversationId}`);
  ack?.({ ok: true });
});

socket.on('chat:message', async ({ conversationId, text }, ack) => {
  if (!text?.trim() || text.length > 2000) return ack?.({ error:
'INVALID' });
  if (!(await chatService.isParticipant(conversationId, socket.userId)))
return ack?.({ error: 'FORBIDDEN' });
  const msg = await chatService.save({ conversationId, sender:
socket.userId, text });
  io.to(`conversation:${conversationId}`).emit('chat:message', msg);
  ack?.({ ok: true, id: msg._id });
  notifyOfflineParticipants(conversationId, msg);
});
```

Notification service

```
export async function notify(userId, { type, title, body, data }) {
  const n = await Notification.create({ user: userId, type, title, body,
data });
  io.to(`user:${userId}`).emit('notification:new', n);
  if (!(await isOnline(userId))) emailQueue.add('send', { userId, title,
body });
  return n;
}
```

Triggered by: new invite/application, request accepted/declined, new chat message (offline), task assigned, deadline approaching, rating requested.

REST endpoints

`GET /conversations`, `GET /conversations/:id/messages?before=<cursor>&limit=30`, `GET /notifications`, `PATCH /notifications/:id/read`, `PATCH /notifications/read-all`.

Frontend

- `socket.js` creates one connection after login with `auth: { token }`; reconnect with the fresh token after refresh.
- `NotificationBell` listens to `notification:new`, increments the badge, shows a toast.
- Chat: virtualized message list, optimistic send, typing indicator (throttled), unread counts.

Phase 6 – Task Management

Method	Route	Description
GET	`/teams/:id/tasks`	List tasks
POST	`/teams/:id/tasks`	Create
PATCH	`/tasks/:id`	Update (status/assignee/due date/order)
DELETE	`/tasks/:id`	Delete
POST	`/tasks/:id/comments`	Comment

- Authorization: only team members.
- After each mutation, emit `task:updated` to `team:<id>`; assigned user also gets a notification.
- Frontend: Kanban with `@hello-pangea/dnd` (To do / In progress / Review / Done); on drop, PATCH status + order optimistically, roll back on error.
- Cron job: daily reminder for tasks due within 24 h.

Phase 7 – Ratings & Admin

Ratings

- Allowed only when `project.status === 'completed'` and both users were on the team; unique per `(project, rater, ratee)`.
- On create, recompute reputation:

```
const agg = await Rating.aggregate([
  { $match: { ratee: userId } },
  { $group: { _id: '$ratee', avg: { $avg: { $avg:
['$scores.teamwork','$scores.communication','$scores.skill','$scores.rel
iability'] } }, count: { $sum: 1 } } },
]);
await User.updateOne({ _id: userId }, { reputation: { avg: agg[0].avg,
count: agg[0].count } });
```

- `POST /ratings`, `GET /users/:id/ratings`; reminder notification when the project completes.

Admin dashboard

Route	Purpose
`GET /admin/stats`	Totals, new users/week, active teams, acceptance rate
`GET/PATCH /admin/users`	List, search, deactivate/reactivate
`GET/PATCH /admin/projects`	Flag/remove listings
`GET/PATCH /admin/reports`	Review and resolve reports
`CRUD /admin/skills`	Manage taxonomy
`GET /admin/audit-log`	Admin actions

All guarded by `requireAuth, requireRole('admin')`; write an `AuditLog` entry for every mutation. Charts via Recharts.

Phase 8 – Hardening, Testing & Deployment

Testing checklist

- [ ] Unit: scoring functions, token utils, validators
- [ ] Integration: register → verify → login → profile → project → invite → accept
- [ ] Authorization: non-members cannot read chat/tasks; non-owners cannot edit projects
- [ ] Socket: unauthorized connection rejected; messages isolated per room
- [ ] E2E (Playwright): main user journey
- [ ] Load test chat and `/matching` (k6/Artillery)

Security checklist

- [ ] Rate limits on `/auth/*` and `/matching/*`
- [ ] Refresh token rotation, httpOnly cookie
- [ ] Input validation on every route
- [ ] CORS limited to the client origin
- [ ] No secrets in repo; `.env.example` only
- [ ] `npm audit` clean

Deployment

Frontend (Vercel)

1. Import repo, root directory `client`, framework Vite.
2. Env: `VITE_API_URL`, `VITE_SOCKET_URL`.
3. Add `vercel.json` rewrite for SPA routing:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Backend (Render)

1. New Web Service, root `server`, build `npm install`, start `npm start`, health check `/health`.
2. Env vars from `.env.example`; `CLIENT_URL` = Vercel URL.
3. MongoDB Atlas (allow Render IPs), Redis via Upstash.
4. Cookies cross-site: set `SameSite=None; Secure` and `app.set('trust proxy', 1)` in production.

AWS alternative: Dockerize the server → ECR → ECS Fargate behind an ALB (enable stickiness), Secrets Manager for env, CloudWatch logs.

CI (`.github/workflows/ci.yml`): install → lint → test → build for both apps on every PR; deploy on merge to `main`.

Demo Script (for evaluation)

1. Register two or three students, complete profiles with different skills.
2. Create a hackathon project that needs frontend, backend and ML roles.
3. Open **Recommendations** → show scores and the "why this match" explanation.
4. Send an invite → receive the real-time notification on the other account → accept.
5. Open the team workspace → chat live, create and drag tasks.
6. Mark the project completed → exchange ratings → see updated reputation.
7. Switch to the admin account → stats, moderation, skill management.

Possible Extensions

- GitHub/LinkedIn import to auto-fill skills
- Calendar integration and meeting scheduling
- Learned ranking model trained on accept/decline data
- Mobile app (React Native) and push notifications
- College-level leaderboards and badges